

Getting Started

Welcome to the NS-LogKit manual. This guide explains how to install, use, and configure the package-based logger that ships with this repository.

Use the links below to navigate the manual:

- [Installation](#)
- [Usage](#)

Installation

To add NS-LogKit to your Unity project, use the Package Manager by adding the package to `Packages/manifest.json`:

```
{
  "dependencies": {
    "com.ns.log-kit": "https://github.com/nicolas-stephan/NS-LogKit.git"
  }
}
```

Alternatively, pin a specific release tag:

```
"com.ns.log-kit": "https://github.com/nicolas-stephan/NS-LogKit.git#v1.1.4"
```

Requirements

- Unity 2024.1 or newer (package manifest targets `6000.0`).

Compile-time toggles

NS-LogKit uses conditional compilation to remove logging calls when logging is disabled.

- `ENABLE_LOGS` — enables `PackageLogger` log methods.
- `ENABLE_VERBOSE_LOG` — additionally enables `Verbose(...)` methods when `ENABLE_LOGS` is defined.

Define these symbols in Player Settings → Scripting Define Symbols, or in the assembly definition settings for the package consumer assembly.

Usage

NS.LogKit provides a small, focused API for runtime channel-based logging:

- `PackageLogger` — lightweight channel logger used from package code.
- `LogChannel` — runtime representation of a channel with a name, enabled state, and minimum log level.
- `LogChannelRegistry` — global registry for channels and group-based control.
- `LogKitSettings` — a `ScriptableObject` loaded from `Resources/LogKitSettings` that persists runtime channel configuration.

PackageLogger

Obtain a logger for a channel with:

```
private static readonly PackageLogger Log = PackageLogger.For("MyPackage.Physics");
```

Available methods (compiled only when `ENABLE_LOGS` is defined):

- `Info(string message, UnityEngine.Object? context = null)`
- `Info(string format, params object[] args)`
- `Warn(string message, UnityEngine.Object? context = null)`
- `Warn(string format, params object[] args)`
- `Error(string message, UnityEngine.Object? context = null)`
- `Error(string format, params object[] args)`
- `Exception(System.Exception ex, UnityEngine.Object? context = null)`
- `Verbose(string message, UnityEngine.Object? context = null)` — additionally requires `ENABLE_VERBOSE_LOG`
- `Verbose(string format, params object[] args)` — additionally requires `ENABLE_VERBOSE_LOG`

Notes

- The logger prefixes messages with the channel name, for example: `[MyPackage.Physics]` message.
- The implementation forwards messages to `UnityEngine.Debug` using `Log`, `LogWarning`, `LogError`, and `LogException`.

Channel control

Use `LogChannelRegistry` to inspect and control channels at runtime:

```
var channel = LogChannelRegistry.GetOrCreate("MyPackage.Physics");
channel.Enable(LogLevel.Info); // enable and set minimum level
channel.Disable(); // disable this channel

// Group operations
LogChannelRegistry.SetGroupEnabled("MyPackage.", false);
LogChannelRegistry.SetGroupMinLevel("MyPackage.", LogLevel.Warning);
LogChannelRegistry.SetAllEnabled(false);
```

LogKitSettings

Create a `LogKitSettings` asset via `Assets -> Create -> NS -> LogKit Settings`.

The asset must be located in a `Resources` folder under the name `LogKitSettings` so it can be loaded automatically at startup. Runtime initialization is handled by the internal `LogSettingsLoader`, which calls `LogKitSettings.LoadAndApply()` during subsystem registration.

`LogKitSettings` applies channel settings as follows:

- if `GlobalEnabled` is false, all logs are disabled;
- otherwise, each `ChannelConfig` entry with a non-empty `Name` is applied to `LogChannelRegistry`.

Each `ChannelConfig` entry exposes:

- `Name`
- `IsEnabled`
- `MinLevel`

Editor integration

The `LogKitSettings` asset exposes `SyncWithRegistry()` under `#if UNITY_EDITOR`, which can be used by editor tooling to populate the settings asset with discovered channels and preserve their current enable/min-level state.